

Word2Vec

In natural language processing, **Word2Vec** is a technique for obtaining vector representations of words. These vectors capture information about the meaning of the word based on the surrounding words, following the principles of distributional semantics. Once trained, the model can be used to find words with similar meanings or usage, while its embeddings can serve as inputs to systems for search and classification.

Word2Vec was developed by Tomáš Mikolov, Kai Chen, Greg Corrado, Ilya Sutskever and Jeff Dean at Google, published in preprints^{[1][2]} and presented at ICLR in 2013.^[3] Its computational efficiency made it practical to learn high-quality word embeddings from very large text corpora and contributed to their widespread adoption in natural language processing.^[4]

Approach

Word2Vec is a group of related models that are used to produce word embeddings. These models are shallow, two-layer neural networks that are trained to reconstruct linguistic contexts of words. Word2Vec takes as its input a large corpus of text and produces a mapping of the set of words to a vector space, typically of several hundred dimensions, with each unique word in the corpus being assigned a vector in the space.

Word2Vec can use either of two model architectures to produce these distributed representations of words: continuous bag of words (CBOW) or continuously sliding skip-gram. In both architectures, Word2Vec considers both individual words and a sliding context window as it iterates over the corpus.

The CBOW can be viewed as a 'fill in the blank' task, where the word embedding represents the way the word influences the relative probabilities of other words in the context window. Words which are semantically similar should influence these probabilities in similar ways, because semantically similar words should be used in similar contexts. The order of context words does not influence prediction (bag of words assumption).

In the continuous skip-gram architecture, the model uses the current word to predict the surrounding window of context words.^{[1][2]} The skip-gram architecture weighs nearby context words more heavily than more distant context words. According to the authors' note,^[5] CBOW is faster while skip-gram does a better job for infrequent words.

After the model is trained, the learned word embeddings are positioned in the vector space such that words that share common contexts in the corpus — that is, words that are semantically and syntactically similar — are located close to one another in the space.^[1] More dissimilar words are located farther from one another in the space.^[1]

Mathematical details

This section is based on expositions.^{[6][7]}

A corpus is a sequence of words. Both CBOW and skip-gram are methods to learn one vector per word appearing in the corpus.

Let V ("vocabulary") be the set of all words appearing in the corpus C . Our goal is to learn one vector $v_w \in \mathbb{R}^d$ for each word $w \in V$.

The idea of skip-gram is that the vector of a word should be close to the vector of each of its neighbors. The idea of CBOW is that the vector-sum of a word's neighbors should be close to the vector of the word.

Continuous bag-of-words (CBOW)

The idea of CBOW is to represent each word with a vector, such that it is possible to predict a word using the sum of the vectors of its neighbors. Specifically, for each word w_i in the corpus, the one-hot encoding of the word is used as the input to the neural network. The output of the neural network is a probability distribution over the dictionary, representing a prediction of individual words in the neighborhood of w_i . The objective of training is to maximize $\sum_i \ln \Pr(w_i \mid w_{i+j}: j \in N)$ where N is a set of (non-zero) indices representing the relative locations of nearby words considered to be in w_i 's neighborhood.

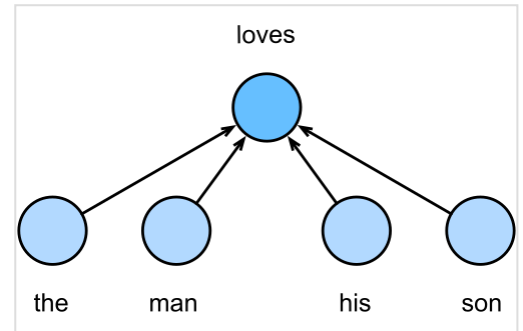
For example, if we want each word in the corpus to be predicted by every other word in a small span of 4 words. The set of relative indexes of neighbor words will be: $N = \{-2, -1, +1, +2\}$, and the objective is to maximize $\sum_i \ln \Pr(w_i \mid w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2})$.

In standard bag-of-words, a word's context is represented by a word-count (aka a word histrogram) of its neighboring words. For example, the "sat" in "the cat sat on the mat" is represented as {"the": 2, "cat": 1, "on": 1}. Note that the last word "mat" is not used to represent "sat", because it is outside the neighborhood $N = \{-2, -1, +1, +2\}$.

In continuous bag-of-words, the histogram is multiplied by a matrix V to obtain a continuous representation of the word's context. The matrix V is also called a **dictionary**. Its columns are the word vectors. It has D columns, where D is the size of the dictionary. Let d be the length of each word vector. We have $V \in \mathbb{R}^{d \times D}$.

For example, multiplying the word histogram {"the": 2, "cat": 1, "on": 1} with V , we obtain $2v_{\text{the}} + v_{\text{cat}} + v_{\text{on}}$.

This is then multiplied with another matrix V' of shape $\mathbb{R}^{D \times d}$. Each row of it is a word vector v' . This results in a vector of length D , one entry per dictionary entry. Then, apply the softmax to obtain a probability distribution over the dictionary.



Continuous bag-of-words (CBOW) model

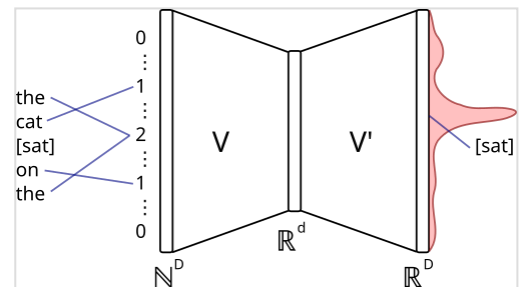


Illustration of CBOW as a neural network

This system can be visualized as a neural network, similar in spirit to an autoencoder, of architecture linear-linear-softmax, as depicted in the diagram. The system is trained by gradient descent to minimize the cross-entropy loss.

In full formula, the cross-entropy loss is:

$$-\sum_i \ln \frac{e^{v'_{w_i} \cdot (\sum_{j \in N} v_{w_{j+i}})}}{\sum_{w'} e^{v'_{w'} \cdot (\sum_{j \in N} v_{w'_{j+i}})}}$$

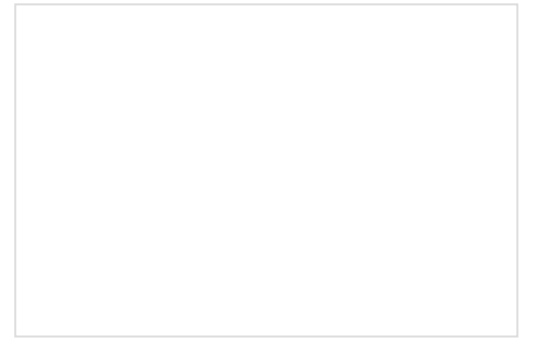
where the outer summation $\sum_i$ is over the words in a corpus, the quantity $\sum_{j \in N} v_{w_{j+i}}$ is the sum of a word's neighbors' vectors, etc.

Once such a system is trained, we have two trained matrices V, V' . Either the column vectors of V or the row vectors of V' can serve as the dictionary. For example, the word "sat" can be represented as either the "sat"-th column of V or the "sat"-th row of V' . It is also possible to simply define $V' = V^\top$, in which case there would no longer be a choice.

Skip-gram

The idea of skip-gram is to represent each word with a vector, such that it is possible to predict the vectors of its neighbors using the vector of a word.

The architecture is still linear-linear-softmax, the same as CBOW, but the input and the output are switched. Specifically, for each word w_i in the corpus, the one-hot encoding of the word is used as the input to the neural network. The output of the neural network is a probability distribution over the dictionary, representing a prediction of individual words in the neighborhood of w_i . The objective of training is to maximize $\sum_i \sum_{j \in N} \ln \Pr(w_{j+i} | w_i)$.



Skip-gram

In full formula, the loss function is

$$-\sum_i \sum_{j \in N} \ln \frac{e^{v'_{w_{j+i}} \cdot v_{w_i}}}{\sum_{w'} e^{v'_{w'} \cdot v_{w_i}}}$$

Same as CBOW, once such a system is trained, we have two trained matrices V, V' . Either the column vectors of V or the row vectors of V' can serve as the dictionary. It is also possible to simply define $V' = V^\top$, in which case there would no longer be a choice.

Essentially, skip-gram and CBOW are exactly the same in architecture. They only differ in the objective function during training.

History

Research on neural language models emerged during the 1980s.^[8] In 2003, Yoshua Bengio and colleagues introduced a neural probabilistic language model that jointly learned distributed word representations and probabilities over word sequences.^[9] In 2010, Tomáš Mikolov and colleagues introduced a recurrent neural-network language model for speech recognition that significantly improved performance over conventional n-gram language models.^[10]

Word2Vec was introduced in two 2013 papers by Tomáš Mikolov and colleagues at Google. The first described the continuous bag-of-words and skip-gram architectures for efficiently learning word representations from large text corpora.^[11] A follow-up paper introduced techniques including negative sampling, sub-sampling of frequent words, and representations of multi-word phrases.^[12] The follow-up paper received the 2013 NeurIPS Test of Time Award.^[13]

In 2017, researchers at Facebook AI Research described an extension of the skip-gram model used in fastText, in which words are represented as combinations of character n-grams. This incorporates subword information into word representations and allows vectors to be constructed for words that did not appear in the training data.^[14]

Static embedding methods such as Word2Vec and fastText produce context-independent word representations. Contextual representations were later developed to account for how word meaning varies with surrounding text, including the recurrent ELMo model and transformer-based models such as BERT.^{[15][16]}

Parameterization

Results of Word2Vec training can be sensitive to parametrization. The following are some important parameters in Word2Vec training.

Training algorithm

A Word2Vec model can be trained with hierarchical softmax and/or negative sampling. To approximate the conditional log-likelihood a model seeks to maximize, the hierarchical softmax method uses a Huffman tree to reduce calculation. The negative sampling method, on the other hand, approaches the maximization problem by minimizing the log-likelihood of sampled negative instances. According to the authors, hierarchical softmax works better for infrequent words while negative sampling works better for frequent words and better with low dimensional vectors.^[5] As training epochs increase, hierarchical softmax stops being useful.^[17]

Sub-sampling

High-frequency and low-frequency words often provide little information. Words with a frequency above a certain threshold, or below a certain threshold, may be subsampled or removed to speed up training.^[18]

Dimensionality

Quality of word embedding increases with higher dimensionality. But after reaching some point, marginal gain diminishes.^[1] Typically, the dimensionality of the vectors is set to be between 100 and 1,000.

Context window

The size of the context window determines how many words before and after a given word are included as context words of the given word. According to the authors' note, the recommended value is 10 for skip-gram and 5 for CBOW.^[5]

Extensions

There are a variety of extensions to Word2Vec.

doc2vec

doc2vec, generates distributed representations of *variable-length* pieces of texts, such as sentences, paragraphs, or entire documents.^{[19][20]} doc2vec has been implemented in the C, Python and Java/Scala tools (see below), with the Java and Python versions also supporting inference of document embeddings on new, unseen documents.

doc2vec estimates the distributed representations of documents much like how Word2Vec estimates representations of words: doc2vec utilizes either of two model architectures, both of which are allegories to the architectures used in Word2Vec. The first, Distributed Memory Model of Paragraph Vectors (PV-DM), is identical to CBOW other than it also provides a unique document identifier as a piece of additional context. The second architecture, Distributed Bag of Words version of Paragraph Vector (PV-DBOW), is identical to the skip-gram model except that it attempts to predict the window of surrounding context words from the paragraph identifier instead of the current word.^[19]

doc2vec also has the ability to capture the semantic 'meanings' for additional pieces of 'context' around words; doc2vec can estimate the semantic embeddings for speakers or speaker attributes, groups, and periods of time. For example, doc2vec has been used to estimate the political positions of political parties in various Congresses and Parliaments in the U.S. and U.K.,^[21] respectively, and various governmental institutions.^[22]

top2vec

Another extension of Word2Vec is top2vec, which leverages both document and word embeddings to estimate distributed representations of topics.^{[23][24]} top2vec takes document embeddings learned from a doc2vec model and reduces them into a lower dimension (typically using UMAP). The space of documents is then scanned using HDBSCAN,^[25] and clusters of similar documents are found. Next, the centroid of documents identified in a cluster is considered to be that cluster's topic vector. Finally, top2vec searches the semantic space for word embeddings located near to the topic vector to ascertain the 'meaning' of the topic.^[23] The word with embeddings most similar to the topic vector might be assigned as the topic's title, whereas far away word embeddings may be considered unrelated.

As opposed to other topic models such as LDA, top2vec provides canonical 'distance' metrics between two topics, or between a topic and another embeddings (word, document, or otherwise). Together with results from HDBSCAN, users can generate topic hierarchies, or groups of related topics and subtopics.

Furthermore, a user can use the results of top2vec to infer the topics of out-of-sample documents. After inferring the embedding for a new document, must only search the space of topics for the closest topic vector.

BioVectors

An extension of word vectors for n-grams in biological sequences (e.g. DNA, RNA, and proteins) for bioinformatics applications has been proposed by Asgari and Mofrad.^[26] Named bio-vectors (BioVec) to refer to biological sequences in general with protein-vectors (ProtVec) for proteins (amino-acid sequences) and gene-vectors (GeneVec) for gene sequences, this representation can be widely used in applications of machine learning in proteomics and genomics. The results suggest that BioVectors can characterize biological sequences in terms of biochemical and biophysical interpretations of the underlying patterns.^[26] A similar variant, dna2vec, has shown that there is correlation between Needleman–Wunsch similarity score and cosine similarity of dna2vec word vectors.^[27]

Radiology and intelligent word embeddings (IWE)

An extension of word vectors for creating a dense vector representation of unstructured radiology reports has been proposed by Banerjee et al.^[28] One of the biggest challenges with Word2Vec is how to handle unknown or out-of-vocabulary (OOV) words and morphologically similar words. If the Word2Vec model has not encountered a particular word before, it will be forced to use a random vector, which is generally far from its ideal representation. This can particularly be an issue in domains like medicine where synonyms and related words can be used depending on the preferred style of radiologist, and words may have been used infrequently in a large corpus.

IWE combines Word2Vec with a semantic dictionary mapping technique to tackle the major challenges of information extraction from clinical texts, which include ambiguity of free text narrative style, lexical variations, use of ungrammatical and telegraphic phases, arbitrary ordering of words, and frequent appearance of abbreviations and acronyms. Of particular interest, the IWE model (trained on the one institutional dataset) successfully translated to a different institutional dataset which demonstrates good generalizability of the approach across institutions.

Analysis

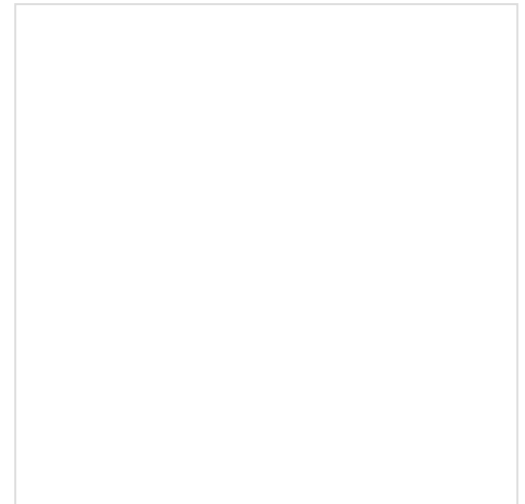
The reasons for successful word embedding learning in the Word2Vec framework are poorly understood. Goldberg and Levy point out that the Word2Vec objective function causes words that occur in similar contexts to have similar embeddings (as measured by cosine similarity) and note that this is in line with J. R. Firth's distributional hypothesis. However, they note that this explanation is "very hand-wavy" and argue that a more formal explanation would be preferable.^[6]

Levy et al. (2015)^[29] show that much of the superior performance of Word2Vec or similar embeddings in downstream tasks is not a result of the models per se, but of the choice of specific hyperparameters. Transferring these hyperparameters to more 'traditional' approaches yields

similar performances in downstream tasks. Arora et al. (2016)^[30] explain Word2Vec and related algorithms as performing inference for a simple generative model for text, which involves a random walk generation process based upon loglinear topic model. They use this to explain some properties of word embeddings, including their use to solve analogies.

Preservation of semantic and syntactic relationships

The word embedding approach is able to capture multiple different degrees of similarity between words. Mikolov et al. (2013)^[31] found that semantic and syntactic patterns can be reproduced using vector arithmetic. Patterns such as "Man is to Woman as Brother is to Sister" can be generated through algebraic operations on the vector representations of these words such that the vector representation of "Brother" - "Man" + "Woman" produces a result which is closest to the vector representation of "Sister" in the model. Such relationships can be generated for a range of semantic relations (such as Country–Capital) as well as syntactic relations (e.g. present tense–past tense).



Visual illustration of word embeddings

This facet of Word2Vec has been exploited in a variety of other contexts. For example, Word2Vec has been used to map a vector space of words in one language to a vector space constructed from another language. Relationships between translated words in both spaces can be used to assist with machine translation of new words.^[32]

Assessing the quality of a model

Mikolov et al. (2013)^[1] developed an approach to assessing the quality of a Word2Vec model which draws on the semantic and syntactic patterns discussed above. They developed a set of 8,869 semantic relations and 10,675 syntactic relations which they use as a benchmark to test the accuracy of a model. When assessing the quality of a vector model, a user may draw on this accuracy test which is implemented in Word2Vec,^[33] or develop their own test set which is meaningful to the corpora which make up the model. This approach offers a more challenging test than simply arguing that the words most similar to a given test word are intuitively plausible.^[1]

Parameters and model quality

The use of different model parameters and different corpus sizes can greatly affect the quality of a Word2Vec model. Accuracy can be improved in a number of ways, including the choice of model architecture (CBOW or Skip-Gram), increasing the training data set, increasing the number of vector dimensions, and increasing the window size of words considered by the algorithm. Each of these improvements comes with the cost of increased computational complexity and therefore increased model generation time.^[1]

In models using large corpora and a high number of dimensions, the skip-gram model yields the highest overall accuracy, and consistently produces the highest accuracy on semantic relationships, as well as yielding the highest syntactic accuracy in most cases. However, the CBOW is less computationally expensive and yields similar accuracy results.^[1]

Overall, accuracy increases with the number of words used and the number of dimensions. Mikolov et al.^[1] report that doubling the amount of training data results in an increase in computational complexity equivalent to doubling the number of vector dimensions.

Altszyler and coauthors (2017) studied Word2Vec performance in two semantic tests for different corpus size.^[34] They found that Word2Vec has a steep learning curve, outperforming another word-embedding technique, latent semantic analysis (LSA), when it is trained with medium to large corpus size (more than 10 million words). However, with a small training corpus, LSA showed better performance. Additionally they show that the best parameter setting depends on the task and the training corpus. Nevertheless, for skip-gram models trained in medium size corpora, with 50 dimensions, a window size of 15 and 10 negative samples seems to be a good parameter setting.

See also

- | | |
|---|--|
| ▪ <u>Autoencoder</u> | ▪ <u>Thought vector</u> |
| ▪ <u>Document-term matrix</u> | ▪ <u>fastText</u> |
| ▪ <u>Feature extraction</u> | ▪ <u>GloVe</u> |
| ▪ <u>Feature learning</u> | ▪ <u>ELMo</u> |
| ▪ <u>Language model § Neural models</u> | ▪ <u>BERT (language model)</u> |
| ▪ <u>Vector space model</u> | ▪ <u>Normalized compression distance</u> |

References

1. Mikolov, Tomas; Chen, Kai; Corrado, Greg; Dean, Jeffrey (16 January 2013). "Efficient Estimation of Word Representations in Vector Space". [arXiv:1301.3781](https://arxiv.org/abs/1301.3781) (<https://arxiv.org/abs/1301.3781>) [cs.CL (<https://arxiv.org/archive/cs>.CL)].
2. Mikolov, Tomas; Sutskever, Ilya; Chen, Kai; Corrado, Greg S.; Dean, Jeff (2013). *Distributed representations of words and phrases and their compositionality*. Advances in Neural Information Processing Systems. [arXiv:1310.4546](https://arxiv.org/abs/1310.4546) (<https://arxiv.org/abs/1310.4546>). Bibcode:2013arXiv1310.4546M (<https://ui.adsabs.harvard.edu/abs/2013arXiv1310.4546M>).
3. "Conference Program - International Conference on Learning Representations 2013" (<https://iclr.cc/archive/2013/program-details/program.html>). *iclr.cc*. Retrieved 23 August 2026.
4. Mikolov, Tomas; Chen, Kai; Corrado, Greg; Dean, Jeffrey (2013). "Efficient Estimation of Word Representations in Vector Space". [arXiv:1301.3781](https://arxiv.org/abs/1301.3781) (<https://arxiv.org/abs/1301.3781>) [cs.CL (<https://arxiv.org/archive/cs>.CL)].
5. "Google Code Archive - Long-term storage for Google Code Project Hosting" (<https://code.google.com/archive/p/Word2Vec/>). *code.google.com*. Retrieved 13 June 2016.
6. Goldberg, Yoav; Levy, Omer (2014). "Word2Vec Explained: Deriving Mikolov et al.'s Negative-Sampling Word-Embedding Method". [arXiv:1402.3722](https://arxiv.org/abs/1402.3722) (<https://arxiv.org/abs/1402.3722>) [cs.CL (<https://arxiv.org/archive/cs>.CL)].
7. Rong, Xin (5 June 2016), *Word2Vec Parameter Learning Explained*, [arXiv:1411.2738](https://arxiv.org/abs/1411.2738) (<https://arxiv.org/abs/1411.2738>)

8. Rumelhart, David E.; McClelland, James L. (1985). On Learning the Past Tenses of English Verbs (<https://eric.ed.gov/?id=ED267419>) (Report). Institute for Cognitive Science, University of California, San Diego.
9. Bengio, Yoshua; Ducharme, Réjean; Vincent, Pascal; Jauvin, Christian (2003). "A Neural Probabilistic Language Model" (<https://www.jmlr.org/papers/v3/bengio03a.html>). *Journal of Machine Learning Research*. **3**: 1137–1155.
10. Mikolov, Tomáš; Karafiát, Martin; Burget, Lukáš; Černocký, Jan; Khudanpur, Sanjeev (2010). "Recurrent Neural Network Based Language Model" (https://www.isca-archive.org/interspeech_2010/mikolov10_interspeech.html). *Proceedings of Interspeech 2010*. International Speech Communication Association. pp. 1045–1048.
11. Mikolov, Tomáš; Chen, Kai; Corrado, Greg; Dean, Jeffrey (2013). "Efficient Estimation of Word Representations in Vector Space". arXiv:1301.3781 (<https://arxiv.org/abs/1301.3781>) [cs.CL (<https://arxiv.org/archive/cs.CL>)].
12. Mikolov, Tomáš; Sutskever, Ilya; Chen, Kai; Corrado, Greg; Dean, Jeffrey (2013). "Distributed Representations of Words and Phrases and their Compositionality" (<https://proceedings.neurips.cc/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html>). *Advances in Neural Information Processing Systems*. Vol. 26. pp. 3111–3119.
13. "Announcing the NeurIPS 2023 Paper Awards" (<https://blog.neurips.cc/2023/12/11/announcing-the-neurips-2023-paper-awards/>). *NeurIPS Blog*. 11 December 2023. Retrieved 6 August 2026.
14. Bojanowski, Piotr; Grave, Edouard; Joulin, Armand; Mikolov, Tomas (2017). "Enriching Word Vectors with Subword Information" (<https://aclanthology.org/Q17-1010/>). *Transactions of the Association for Computational Linguistics*. **5**: 135–146. doi:10.1162/tacl_a_00051 (https://doi.org/10.1162%2Ftacl_a_00051).
15. Peters, Matthew E.; Neumann, Mark; Iyyer, Mohit; Gardner, Matt; Clark, Christopher; Lee, Kenton; Zettlemoyer, Luke (2018). "Deep Contextualized Word Representations" (<https://aclanthology.org/N18-1202/>). *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. pp. 2227–2237. arXiv:1802.05365 (<https://arxiv.org/abs/1802.05365>). doi:10.18653/v1/N18-1202 (<https://doi.org/10.18653%2Fv1%2FN18-1202>).
16. Devlin, Jacob; Chang, Ming-Wei; Lee, Kenton; Toutanova, Kristina (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" (<https://aclanthology.org/N19-1423/>). *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. pp. 4171–4186. doi:10.18653/v1/N19-1423 (<https://doi.org/10.18653%2Fv1%2FN19-1423>).
17. "Parameter (hs & negative)" (<https://groups.google.com/forum/#!msg/Word2Vec-toolkit/WUWad9fL0jU/LdbWy1jQjUIJ>). *Google Groups*. Retrieved 13 June 2016.
18. "Visualizing Data using t-SNE" (<http://jmlr.csail.mit.edu/papers/volume9/vandermaaten08a/vandermaaten08a.pdf>) (PDF). *Journal of Machine Learning Research*, 2008, vol. 9, p. 2595. Retrieved 18 March 2017.
19. Le, Quoc; Mikolov, Tomas (May 2014). "Distributed Representations of Sentences and Documents". *Proceedings of the 31st International Conference on Machine Learning*. arXiv:1405.4053 (<https://arxiv.org/abs/1405.4053>).
20. Rehurek, Radim. "Gensim" (<https://radimrehurek.com/gensim/models/doc2vec.html>).
21. Rheault, Ludovic; Cochrane, Christopher (3 July 2019). "Word Embeddings for the Analysis of Ideological Placement in Parliamentary Corpora" (<https://www.cambridge.org/core/journals/political-analysis/article/word-embeddings-for-the-analysis-of-ideological-placement-in-parliamentary-corpora/017F0CEA9B3DB6E1B94AC36A509A8A7B>). *Political Analysis*. **28** (1).
22. Nay, John (21 December 2017). "Gov2Vec: Learning Distributed Representations of Institutions and Their Legal Text" (https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3087278). SSRN. arXiv:1609.06616 (<https://arxiv.org/abs/1609.06616>). SSRN 3087278 (https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3087278).
23. Angelov, Dimo (August 2020). "Top2Vec: Distributed Representations of Topics". arXiv:2008.09470 (<https://arxiv.org/abs/2008.09470>) [cs.CL (<https://arxiv.org/archive/cs.CL>)].

24. Angelov, Dimo (11 November 2022). "Top2Vec" (<https://github.com/ddangelov/Top2Vec>). *GitHub*.
25. Campello, Ricardo; Moulavi, Davoud; Sander, Joerg (2013). "Density-Based Clustering Based on Hierarchical Density Estimates" (https://link.springer.com/chapter/10.1007/978-3-642-37456-2_14). *Advances in Knowledge Discovery and Data Mining*. Lecture Notes in Computer Science. Vol. 7819. pp. 160–172. doi:10.1007/978-3-642-37456-2_14 (https://doi.org/10.1007/978-3-642-37456-2_14). ISBN 978-3-642-37455-5.
26. Asgari, Ehsaneddin; Mofrad, Mohammad R.K. (2015). "Continuous Distributed Representation of Biological Sequences for Deep Proteomics and Genomics" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4640716>). *PLOS ONE*. **10** (11) e0141287. arXiv:1503.05140 (<https://arxiv.org/abs/1503.05140>). Bibcode:2015PLoS...1041287A (<https://ui.adsabs.harvard.edu/abs/2015PLoSO..1041287A>). doi:10.1371/journal.pone.0141287 (<https://doi.org/10.1371/journal.pone.0141287>). PMC 4640716 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4640716>). PMID 26555596 (<https://pubmed.ncbi.nlm.nih.gov/26555596>).
27. Ng, Patrick (2017). "dna2vec: Consistent vector representations of variable-length k-mers". arXiv:1701.06279 (<https://arxiv.org/abs/1701.06279>) [q-bio.QM (<https://arxiv.org/archive/q-bio.QM>)].
28. Banerjee, Imon; Chen, Matthew C.; Lungren, Matthew P.; Rubin, Daniel L. (2018). "Radiology report annotation using intelligent word embeddings: Applied to multi-institutional chest CT cohort" (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5771955>). *Journal of Biomedical Informatics*. **77**: 11–20. doi:10.1016/j.jbi.2017.11.012 (<https://doi.org/10.1016/j.jbi.2017.11.012>). PMC 5771955 (<https://www.ncbi.nlm.nih.gov/pmc/articles/PMC5771955>). PMID 29175548 (<https://pubmed.ncbi.nlm.nih.gov/29175548>).
29. Levy, Omer; Goldberg, Yoav; Dagan, Ido (2015). "Improving Distributional Similarity with Lessons Learned from Word Embeddings" (<http://www.aclweb.org/anthology/Q15-1016>). *Transactions of the Association for Computational Linguistics*. **3**: 211–225. doi:10.1162/tacl_a_00134 (https://doi.org/10.1162/tacl_a_00134).
30. Arora, S; et al. (Summer 2016). "A Latent Variable Model Approach to PMI-based Word Embeddings" (<http://aclweb.org/anthology/Q16-1028>). *Transactions of the Association for Computational Linguistics*. **4**: 385–399. arXiv:1502.03520 (<https://arxiv.org/abs/1502.03520>). doi:10.1162/tacl_a_00106 (https://doi.org/10.1162/tacl_a_00106) – via ACLWEB.
31. Mikolov, Tomas; Yih, Wen-tau; Zweig, Geoffrey (2013). "Linguistic Regularities in Continuous Space Word Representations". *HLT-Naacl*: 746–751.
32. Jansen, Stefan (9 May 2017). "Word and Phrase Translation with Word2Vec". arXiv:1705.03127 (<https://arxiv.org/abs/1705.03127>) [cs.CL (<https://arxiv.org/archive/cs.CL>)].
33. "Gensim - Deep learning with Word2Vec" (<https://radimrehurek.com/gensim/models/Word2Vec.html>). Retrieved 10 June 2016.
34. Altszyler, E.; Ribeiro, S.; Sigman, M.; Fernández Slezak, D. (2017). "The interpretation of dream meaning: Resolving ambiguity using Latent Semantic Analysis in a small corpus of text". *Consciousness and Cognition*. **56**: 178–187. arXiv:1610.01520 (<https://arxiv.org/abs/1610.01520>). doi:10.1016/j.concog.2017.09.004 (<https://doi.org/10.1016/j.concog.2017.09.004>). PMID 28943127 (<https://pubmed.ncbi.nlm.nih.gov/28943127>). S2CID 195347873 (<https://api.semanticscholar.org/CorpusID:195347873>).

External links

- [Wikipedia2Vec \(https://wikipedia2vec.github.io/demo/\)](https://wikipedia2vec.github.io/demo/)[1] (introduction (<https://wikipedia2vec.github.io/wikipedia2vec/>))

Implementations

- [C \(https://github.com/tmikolov/Word2Vec\)](https://github.com/tmikolov/Word2Vec)

- C++ (<https://github.com/codehiker42/Word2Vec>)
 - C# (<https://github.com/eabdullin/Word2Vec.Net>)
 - Python (Spark) (<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.feature.Word2Vec.html>)
 - Python (TensorFlow) (https://github.com/tensorflow/tensorflow/blob/r1.1/tensorflow/examples/tutorials/Word2Vec/Word2Vec_basic.py)
 - Python (Gensim) (<http://radimrehurek.com/gensim/models/Word2Vec.html>)
 - Java/Scala (<https://github.com/deeplearning4j/deeplearning4j>)
 - R (<https://github.com/bnosac/Word2Vec>)
-

Retrieved from "<https://en.wikipedia.org/w/index.php?title=Word2Vec&oldid=1375450281>"